Graph Search

Graph traversal

Graph traversal is the process of visiting all vertices reachable from a starting vertex following a well-defined order.

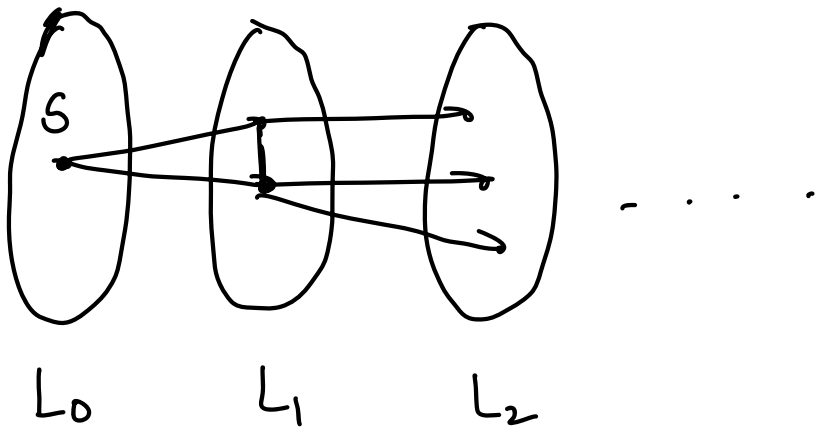
Standard Graph Search Algorithms

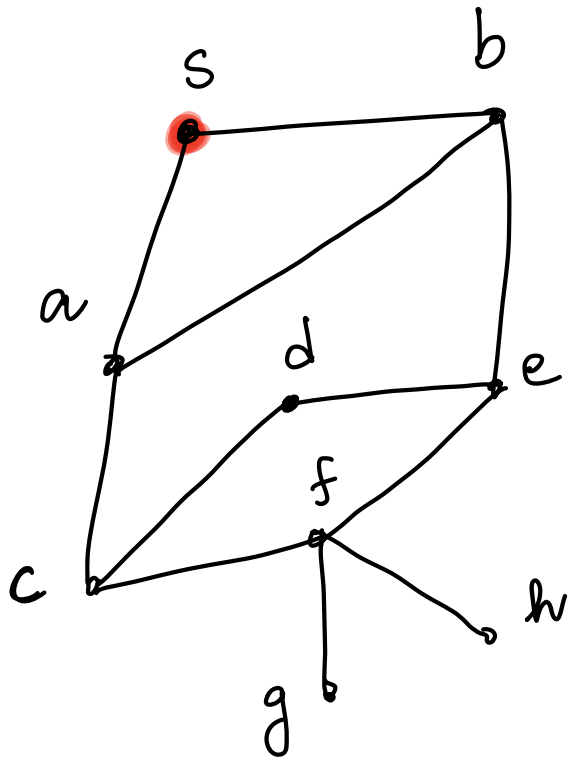
- Breadth First Search (BFS)
- Depth First Search (DFS)

Breadth-first Search

Overview

Explore the vertices of a graph in layers.




s
b
a
d
e
f
c
h
g

Notation:

- $u.color$: Color stored for the vertex u
- $u.\pi$: Predecessor of u. If u has no predecessor then $u.\pi = NIL$.
- $u.d$: distance of u from the source s computed by the algorithm.

The algorithm uses Queue Q (first-in, first out).

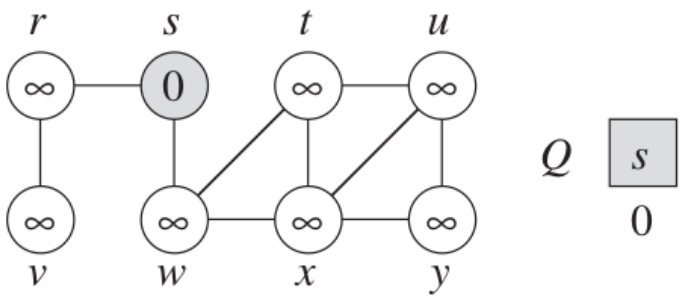
We assume that the input graph $G = (V, E)$ is represented using adjacency lists.

[Source: Coreman's book]

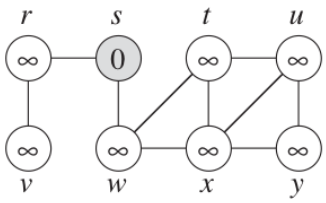
s : Starting vertex.

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for each v ∈ G.Adj[u]
13            if v.color == WHITE
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)
18       u.color = BLACK
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for each v ∈ G.Adj[u]
13            if v.color == WHITE
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)
18       u.color = BLACK
```

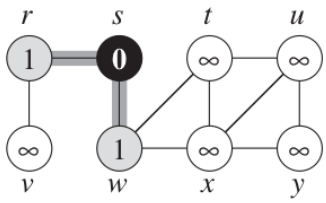


(a)



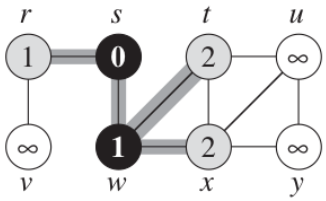
Q | s | | |
0

(b)



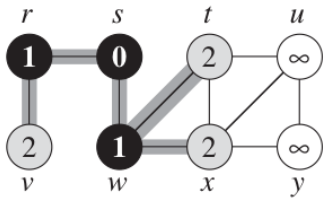
Q | w | r | |
1 1

(c)



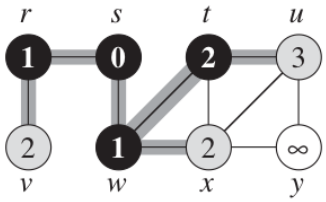
Q | r | t | x |
1 2 2

(d)



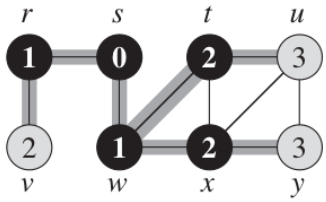
Q | t | x | v |
2 2 2

(e)



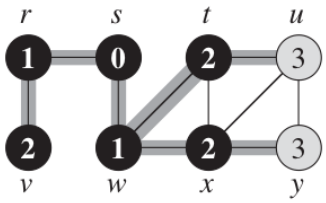
Q | x | v | u |
2 2 3

(f)



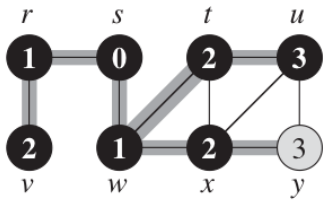
Q | v | u | y |
2 3 3

(g)



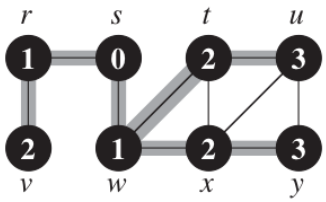
Q | u | y | |
3 3

(h)



Q | y | | |
3

(i)



Q $\emptyset$

Runtime - Analysis:

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY
15              v.d = u.d + 1
16              v.π = u
17              ENQUEUE(Q, v)
18      u.color = BLACK
```

- The BFS algorithm never colors a vertex with White. Thus each vertex is enqueued at most once and hence dequeued at most once.

- Since the algorithm scans the adjacency list of each vertex only when the vertex dequeued, it scans each adjacency list at most once.

- Sum of the lengths of all adjacency list is $\Theta(E)$.

- Overhead initialization is $O(V)$

- Total running time of BFS algo is $O(V+E)$.

Bfs runs in time linear in the size of the adjacency-list representation of G.

Breadth first Search trees

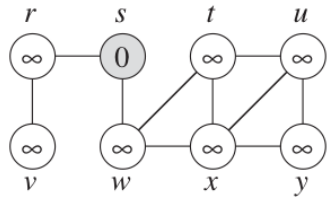
The Procedure BFS builds a BFS tree as it searches the graph.

let $G = (V, E)$ with source s, BFS tree of G is defined as $G_\pi = (V_\pi, E_\pi)$ (also called Predecessor subgraph)

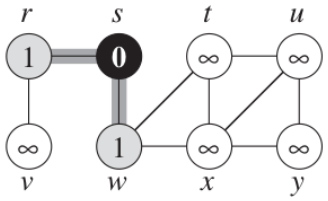
$$V_\pi = \{ v \in V : v.\pi \neq NIL \} \cup \{s\}$$

$$E_\pi = \{ (v.\pi, v) : v \in V_\pi - \{s\} \}$$

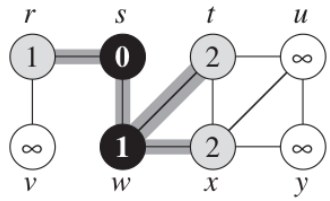
The dark edges indicate BFS tree.

(a) 

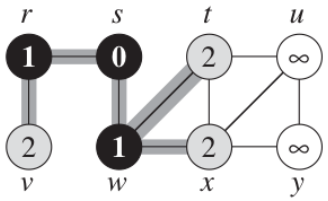
Q: s (0)

(b) 

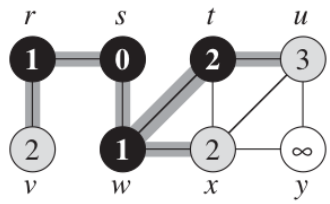
Q: w r (1 1)

(c) 

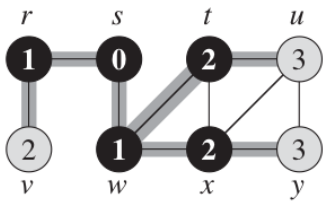
Q: r t x (1 2 2)

(d) 

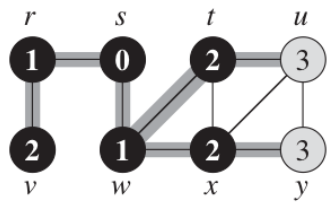
Q: t x v (2 2 2)

(e) 

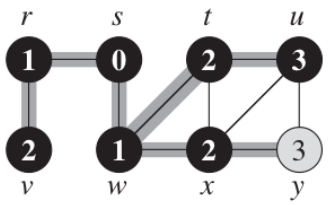
Q: x v u (2 2 3)

(f) 

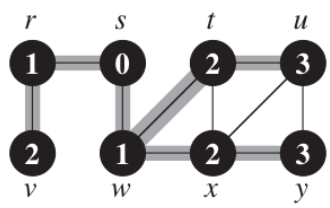
Q: v u y (2 3 3)

(g) 

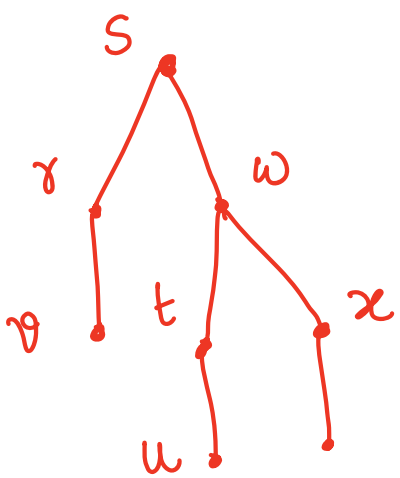
Q: u y (3 3)

(h) 

Q: y (3)

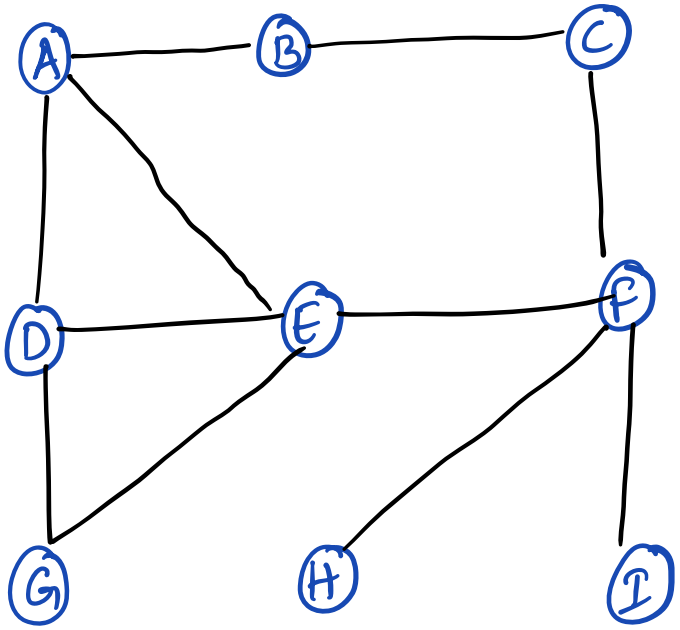
(i) 

Q: $\emptyset$



BFS tree

Quiz:



Perform a BFS from node A, with a preference for visiting lower-character vertices before higher-character vertices.

Applications of BFS

BFS can be used to solve many problems in graph algorithms. For example

- Finding Shortest Path between two vertices [Coreman]
- Testing bipartiteness of a graph [KT: Chapter 3.4]
- Finding the number of connected Components [KT: Chap 3.2]

etc ...

Notation:

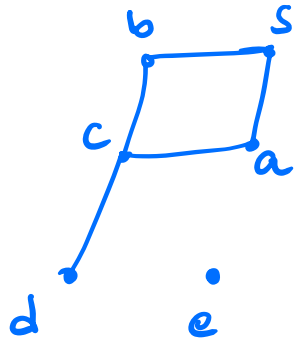
$G = (V, E)$, $s \in V$

$\delta(s, v)$: denotes the Shortest Path distance from s to v.

if there is no Path from s to v then $\delta(s, v) = \infty$

Any path of length $\delta(s, v)$ from s to v is called a Shortest Path.

Eg



$\delta(s, d) = 3$

$\delta(s, e) = \infty$

P: s b c d is a shortest s to d Path.

Theorem: (Correctness of BFS)

Let $G=(V,E)$ be a directed | undirected graph and suppose BFS is run on G from a given source $s \in V$. Then during the execution, BFS discovers every vertex $v \in V$ that is reachable from s and upon termination $v.d = \delta(s,v)$ for all $v \in V$.

Finding number of Connected Components

Def: let $G=(V,E)$ be an undirected graph

A connected component of G is a subgraph $G'=(V',E')$ such that

(a) for every pair of vertices $u, v \in V'$ there exists a path between u and v in G'.

(b) G' is maximal with respect to this property ie, adding any other vertex from $V \setminus V'$ would break the connectivity.

Ex:



G

The graph G has three connected components.

Pseudo Code : Nodes are labelled 1 to n

- Components = 1
- All Nodes unexplored
- For $i = 1$ to n
  - if i not explored
    - BFS (G, i) → finds one connected component.
    - Components = Components + 1

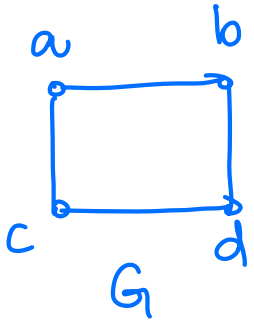
Running time: $O(n+m)$

$$= \sum_i \text{Running time of BFS}(G, i)$$

Bipartite Graphs (Recap)

Def: A graph $G=(V,E)$ is bipartite if its vertex set V can be partitioned into two sets X and Y such that every edge connects one vertex in X to one in Y.

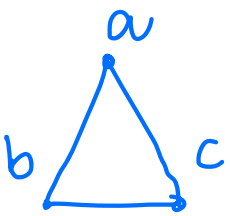
Ex:

① 

$X = \{a, d\}$

$Y = \{b, c\}$

G is bipartite

② 

G is Non-bipartite.

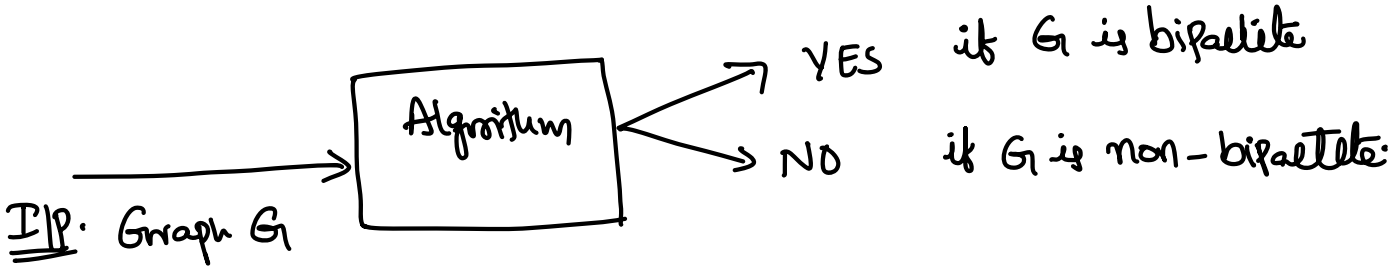
Exercise

Theorem: A graph G is bipartite if and only if G has no odd length cycles.

Testing Bipartiteness : An application of BFS

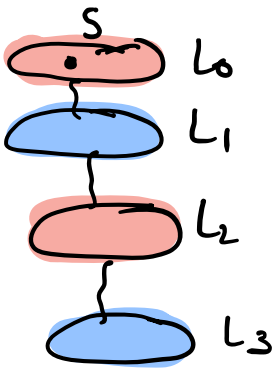
Input : An undirected graph G

Question : Is G Bipartite ?



Simple Algorithm: s: Starting node

We Perform BFS, coloring s red, all of layer L_1 blue, all of layer L_2 red and so on.



Odd numbered layers colored blue

Even numbered layers Color red

We can implement this by adding an extra array Color over the nodes.

Whenever we get to a step in BFS where we are adding a node v to a list $L[i+1]$.

we assign Color[v] = red if $i+1$ is an even number

= blue if " " " odd ".

At the end for every edge $uv \in E(G)$

Check whether both end received the different Color or not.

Total running time = $O(n+m)$

Correctness: [KT - Chapter 3]

Lemma :- Let G be a connected graph and let $L_1, L_2, \ldots$ be the layers produced by BFS starting at node s. Then exactly one of the following two things must hold.

(a) There is no edge of G joining two nodes of the same layer. In this case G is bipartite in which the nodes in even-numbered layers can be colored red and the nodes in odd numbered layers can be colored blue.

(b) There is an edge of G joining two nodes of the same layer. In this case G contains an odd length cycle and so it cannot be bipartite.